OP_UNM / OP_LEN / OP_NOT

今天来追一元操作符的操作，这一篇比较可以说是相当的水了，因为本来也不难

OP_UNM：一元取负

这个的意思是取反

```
L_OP_UNM: {
    StkId ra = RA(i); // A
    TValue *rb = vRB(i); // B值
    lua_Number nb;
    if (ttisinteger(rb)) { // 如果是整数
        lua_Integer ib = ivalue(rb);
        setivalue(s2v(ra), intop(-, 0, ib)); // 取反
    } else if (tonumberns(rb, nb)) { // 转化为浮点数类型
        setfltvalue(s2v(ra), luai_numunm(L, nb)); // 直接对浮点数类型操作
    } else
        Protect(luaT_trybinTM(L, rb, rb, ra, TM_UNM)); // 调用元方法
    vmbreak;
}
```

1. **目标寄存器** `ra = RA(i)`
   - 取出指令编码里的 A 寄存器索引，用来存放结果。
2. **操作数寄存器** `rb = vRB(i)`
   - 取出 B 寄存器（操作对象）。
3. **整数快速路径**
   - `ttisinteger(rb)` → 如果 B 是整数类型
   - `intop(-, 0, ib)` → 做整数取负
   - 写回 `R[A]`
4. **浮点快速路径**
   - `tonumberns(rb, nb)` → 尝试把 B 转成浮点数（number）

- `luai_numunm(L, nb)` → 浮点取负
- 写回 `R[A]`

5. **元方法路径（slow path）**
   - 如果既不是整数也不能转浮点，调用 `luaT_trybinTM`
   - `TM_UNM` 表示元方法 `__unm`

```c
void luaT_trybinTM(lua_State *L, const TValue *p1, const TValue *p2, StkId
res, TMS event)
{
    if (l_unlikely(callbinTM(L, p1, p2, res, event) < 0)) {
        switch (event) {
        case TM_BAND:
        case TM_BOR:
        case TM_BXOR:
        case TM_SHL:
        case TM_SHR:
        case TM_BNOT: {
            if (ttisnumber(p1) && ttisnumber(p2))
                luaG_tointerror(L, p1, p2);
            else
                luaG_opinterror(L, p1, p2, "perform bitwise operation on");
        }
        /* calls never return, but to avoid warnings: */ /* FALLTHROUGH */
        default:
            luaG_opinterror(L, p1, p2, "perform arithmetic on");
        }
    }
}
```

1. `callbinTM` 尝试找到元方法 `TM_UNM` 并调用
2. 返回 `< 0` 表示没有找到元方法
3. 没找到时抛出错误：
   - 位运算类型 → `luaG_opinterror` / `luaG_tointerror`
   - 其他算术 → `luaG_opinterror`

```c
static int callbinTM(lua_State *L, const TValue *p1, const TValue *p2, StkId
res, TMS event)
{
    const TValue *tm = luaT_gettmbyobj(L, p1, event); /* try first operand */
    if (notm(tm))
        tm = luaT_gettmbyobj(L, p2, event); /* try second operand */
    if (notm(tm))
        return -1; /* tag method not found */
    else /* call tag method and return the tag of the result */
        return luaT_callTMres(L, tm, p1, p2, res);
}
```

1. **查找元方法**
   - 优先操作数 `p1` 的 metatable
   - 如果没有，再查 `p2` 的 metatable
2. **没找到元方法** → 返回 -1
3. **找到元方法** → 调用 `luaT_callTMres`，把结果写入 `res`

OP_LEN：获取长度

这个指令是获取当前数据结构的长度，如果是字符串就是字符串的长度，如果是表格就是表格内元素数量，数字不可取此属性

```c
vmcase(OP_LEN)
{
    StkId ra = RA(i);
    Protect(luaV_objlen(L, ra, vRB(i)));
    vmbreak;
}
void luaV_objlen(lua_State *L, StkId ra, const TValue *rb)
{
    const TValue *tm;
    switch (ttypetag(rb)) {
```

```
    case LUA_VTABLE: { // 表
        Table *h = hvalue(rb);
        tm = fasttm(L, h->metatable, TM_LEN);
        if (tm)
            break; /* metamethod? break switch to call it */
        setivalue(s2v(ra), l_castU2S(luaH_getn(L, h))); /* else primitive len
*/
        return;
    }
    case LUA_VSHRSTR: {  // 短字符串
        setivalue(s2v(ra), tsvalue(rb)->shrlen);
        return;
    }
    case LUA_VLNGSTR: { // 长字符串
        setivalue(s2v(ra), cast_st2S(tsvalue(rb)->u.lnglen));
        return;
    }
    default: { /* try metamethod */
        tm = luaT_gettmbyobj(L, rb, TM_LEN); // 获取元方法
        //              校验是不是int类型
        // 先查找是否存在元方法，int类型没有元方法也不属于上述所有的类型，所以这里
直接报错，整数类型也无法取长度
        if (l_unlikely(notm(tm))) /* 调用错误 */
            luaG_typeerror(L, rb, "get length of");
        break;
    }
    }
    luaT_callTMres(L, tm, rb, rb, ra);
}
```

OP_NOT：对象取反

```
vmcase(OP_NOT)
{
    StkId ra = RA(i); // A
    TValue *rb = vRB(i); // B值
    if (l_isfalse(rb)) // 是否直接位false或者nil
        setbtvalue(s2v(ra)); // 设置true
    else
```

```
        setbfvalue(s2v(ra)); // 设置false
    vmbreak;
}
```